

Helm Autonomy and Behavior Properties

How the IvP Helm selects actions and how behavior properties are configured.

Included MIT MOOS-IvP PDF sources:

- chap_helm_autonomy.pdf
- chap_bhv_props.pdf

IvP Helm Autonomy

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_helm_autonomy

1	IvP Helm Autonomy	1
1.1	Overview	1
1.2	Inside the Helm - A Look at the Helm Iterate Loop	4
1.3	Mission Behavior Files	6
1.4	Hierarchical Mode Declarations	8
1.5	Behavior Participation in the IvP Helm	12
1.6	Behavior Reconciliation in the IvP Helm - Multi-Objective Optimization	16

Contents

1 IvP Helm Autonomy

1.1 Overview

An autonomous helm is primarily an engine for decision making. The IvP Helm uses a behavior-based architecture to organize its decision making and is distinctive in the manner in which it resolves competition between competing behaviors - it performs multi-objective optimization on their collective output using a mathematical programming model called interval programming. Here the IvP Helm architecture is described and the means for configuring it given a set of behaviors and a set of mission objectives.

1.1.1 The Influence of Brooks, Stallman and Dantzig on the IvP Helm

The notion of a behavior-based architecture for implementing autonomy on a robot or unmanned vehicle is most often attributed to Rodney Brooks' Subsumption Architecture, [Brooks, 1986]. A key principle at the heart of Brooks' architecture and arguably the primary reason its appeal has endured, is the notion that autonomy systems can be built *incrementally*. Notably, Brooks' original publication pre-dated the arrival of Open Source software and the Free Software Foundation founded by Richard Stallman. Open Source software is not a pre-requisite for building autonomy systems incrementally, but it has the capability of greatly accelerating that objective. The development of complex autonomy systems stands to significantly benefit if the set of developers at the table is large and diverse. Even more so if they can be from different organizations with perhaps even the loosest of overlap in interest regarding how to use the collective end product.

A key issue in behavior-based autonomy has been the issue of action selection, and the IvP Helm is distinct in this regard with the use of multi-objective optimization and interval programming. The algorithm behind interval programming, as well as the term itself, was motivated by the

mathematical programming model, linear programming, developed by George Dantzig, [Dantzig 1948]. The key idea in linear programming is the choice of the particular mathematical construct that comprises an instance of a linear programming problem - it has enough expressive flexibility to represent a huge class of practical problems, *and* the constructs can be effectively exploited by the simplex method to converge quickly even on very large problem instances. The constructs used in interval programming to represent behavior output (piecewise linear functions) were likewise chosen to have enough expressive flexibility to handle any current and future behavior, and due to the opportunity to develop solution algorithms that exploit the piecewise linear constructs.

1.1.2 Traditional and Non-traditional Aspects of the IvP Behavior-Based Helm

The IvP Helm indeed takes its motivation from early notions of the behavior-based architecture, but is also quite different in many regards. The notion of behavior independence to temper the growth of complexity in progressively larger systems is still a principle closely followed in the IvP Helm. Behaviors may certainly influence one another from one iteration to the next, as we'll see in discussions in this section. This was also evident in the Alpha example mission where the completion of the Survey behavior triggered the Return behavior. But within a single iteration, the output generated by a single behavior is not affected at all by what is generated by other behaviors in the same iteration. The only inter-behavior "communication" realized within an iteration comes when the IvP solver reconciles the output of multiple behaviors. The independence of behaviors not only helps a single developer manage the growth of complexity, but it also limits the dependency between developers. A behavior author need not worry that a change in the implementation of another behavior by another author requires subsequent recoding of one's own behavior(s).

Certain aspects of behaviors in the IvP Helm may also be a departure from some notions traditionally associated (fairly or not) with behavior-based architectures:

- Behaviors have state. IvP behaviors are instances of a class with a fairly simple interface to the helm. Inside they may be arbitrarily complex, keep histories of observed sensor data, and may contain algorithms that could be considered "reactive" or "plan-based".
- Behaviors influence each other between iterations. The primary output of behaviors is their objective function, ranking the utility of candidate actions. IvP behaviors may also generate variable-value posts to the **MOOSDB** observable by behaviors on the next helm iteration. In this way they can explicitly influence other behaviors by triggering or suppressing their activation or even affecting the parameter configuration of other behaviors.
- Behaviors may accept externally generated plans. The input to a behavior can be anything represented by a MOOS variable, and perhaps generated by other MOOS processes outside the helm. It is allowable to have one or more planning engines running on the vehicle generating output consumed by one or more behaviors.
- Several instances of the same behavior. Behaviors generally accept a set of configuration parameters that allow them to be configured for quite different tasks or roles in the same helm and mission. Different waypoint behaviors, for example, can be configured for different components of a transit mission. Or different collision avoidance behaviors can be instantiated for different contacts.

- Behaviors can be run in a configurable sequence. Due to the `condition` and `endflag` parameters defined for all behaviors, a sequence of behaviors can be readily configured into a larger mission plan.
- Behaviors rate actions over a coupled decision space. IvP functions generated by behaviors are defined over the Cartesian product of the set of vehicle decision variables. This is distinct from the de-coupled decision making style proposed in [Pirjanian 1998] and [Rosenblatt 1997] - early advocates of multi-objective optimization in behavior-based action selection.

1.1.3 Two Layers of Building Autonomy in the IvP Helm

The autonomy in play on a vehicle during a particular mission is the product of two distinct efforts - (1) the development of vehicle behaviors and their algorithms, and (2) mission planning via the configuration of behaviors and mode declarations. The former involves the writing of new source code, and the latter involves the editing of mission behavior files, such as the simple example for the Alpha example mission.

1.2 Inside the Helm - A Look at the Helm Iterate Loop

Like other MOOS applications, the IvP Helm implements an `Iterate()` loop within which the basic function of the helm is executed. Components of the `Iterate()` loop, with respect to the behavior-based architecture, are described in this section. The basic flow, in five steps, is depicted in Figure 1. Description of the five components follow.

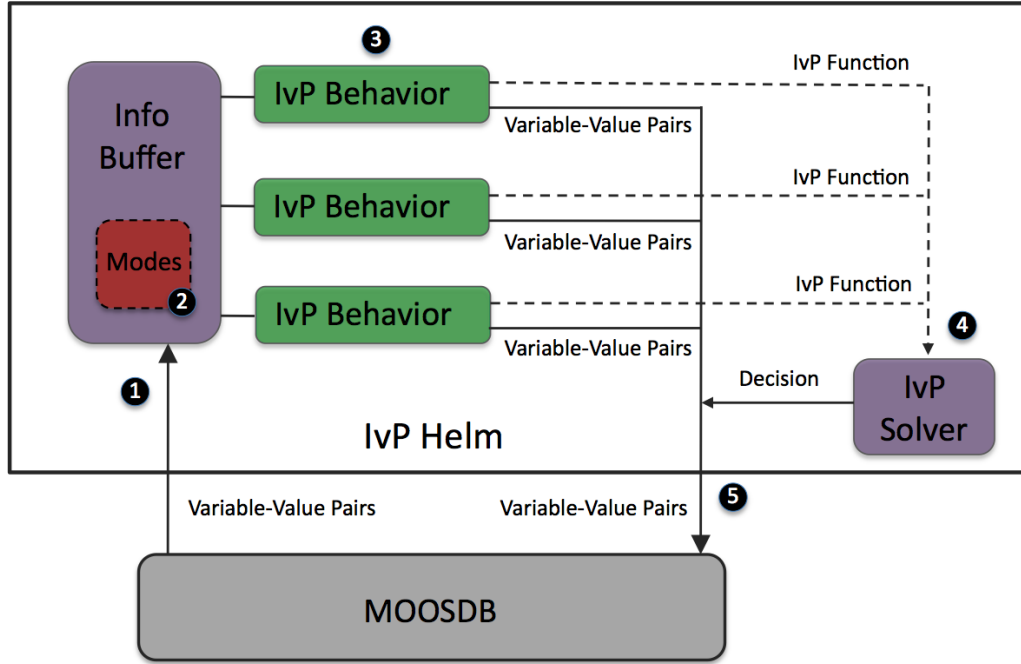


Figure 1: **The pHelmIvP Iterate Loop:** (1) Mail is read from the MOOSDB. It is parsed and stored in a local buffer to be available to the behaviors, (2) If there were any mode declarations in the mission behavior file they are evaluated at this step. (3) Each behavior is queried for its contribution and may produce an IvP function and a list of variable-value pairs to be posted to the MOOSDB at the end of the iteration, (4) the objective functions are resolved to produce an action, expressible as a set of variable-value pairs, (5) all variable-value pairs are published to the MOOSDB for other MOOS processes to consume.

1.2.1 Step 1 - Reading Mail and Populating the Info Buffer

The first step of a helm iteration occurs outside the `Iterate()` loop. As depicted in Figure ??, a MOOS application will read its mail by executing its `OnNewMail()` function just prior to executing its `Iterate()` loop if there is any mail in its in-box. The helm parses mail to maintain its own information buffer which is also a mapping of variables to values. This is done primarily for simplicity - to ensure that each behavior is acting on the same world state as represented by the info buffer. Each behavior has a pointer to the buffer and is able to query the current value of any variable in the buffer, or get a list of variable-value changes since the previous iteration.

1.2.2 Step 2 - Evaluation of Mode Declarations

Once the information buffer is updated with all incoming mail, the helm evaluates any mode declarations specified in the behavior file. Mode declarations are discussed in Section 1.4. In short, a mode is represented by a string variable that is reset on each iteration based on the evaluation of a set of logic expressions involving other variables in the buffer. The variable representing the mode declaration is then available to the behavior on the current iteration when it, for example, evaluates its `condition` parameters. A condition for behavior participating in the current iteration could therefore read something like `condition = (MODE==SURVEYING)`. The exact value of the variable `MODE` is set during this step of the `Iterate()` loop.

1.2.3 Step 3 - Behavior Participation

In the third step much of the work of the helm is realized by giving each behavior a chance to participate. Each behavior is queried sequentially - the helm contains no separate threads in this regard. The order in which behaviors is queried does not affect the output. This step contains two distinct parts for each behavior - (1) Determination of whether the behavior will participate, and (2) production of output if it is indeed participating on this iteration. Each behavior may produce two types of information as the Figure 1 indicates. The first is an objective function (or "utility" function) in the form of an IvP function. The second kind of behavior output is a list of variable-value pairs to be posted by the helm to the MOOSDB at the end of the `Iterate()` loop. A behavior may produce both kinds of information, neither, or one or the other, on any given iteration.

1.2.4 Step 4 - Behavior Reconciliation

In the fourth step depicted in Figure 1, the IvP functions are collected by the IvP solver to produce a single decision over the helm's decision space. Each function is an IvP function - an objective function that maps each element of the helm's decision space to a utility value. In this case the functions are of a particular form - piecewise linearly defined. That is, each piece is an *interval* of the decision space with an associated linear function. Each function also has an associated weight and the solver performs multi-objective optimization over the weighted sum of functions (in effect a single objective optimization at that point). The output is a single optimal point in the decision space. For each decision variable the helm produces another variable-value pair, such as `DESIRED_SPEED = 2.4` for publication to the MOOSDB.

1.2.5 Step 5 - Publishing the Results to the MOOSDB

In the last step, the helm simply publishes all variable-value pairs to the MOOSDB, some of which were produced directly by the behaviors, and some of which were generated as output from the IvP Solver. The helm employs the duplication filter, only on the variable-value pairs generated directly from the behaviors, and not the variable-value pairs generated by the IvP solver that represent a decision in the helm's domain. For example, even if the decision about a vehicle's depth, represented by the variable `DESIRED_DEPTH` produced by the helm were unchanged for 5 minutes of operation, it would be published on each iteration of the helm. To do otherwise could give the impression to consumers of the variable that the variable is "stale", which could trigger an unwanted override of the helm out of concern for safety.

1.3 Mission Behavior Files

The helm is configured for a particular mission primarily through one or more mission behavior files, typically with a *.bhv suffix. Behavior files have three types of entries, usually but not necessarily kept in three distinct parts - (1) variable initializations, (2) behavior configurations, and (3) hierarchical mode declarations. These three parts are discussed below. The example `alpha.bhv` file did not contain hierarchical mode declarations, but does contain examples of variable initializations and behavior configurations.

1.3.1 Variable Initialization Syntax

The syntax for variable initialization is fairly straight-forward:

```
initialize <variable> = <value>
...
initialize <variable> = <value>
```

Multiple initializations may be declared on a single line by separating each variable-value pair with a comma. The keyword `initialize` is case insensitive. The `<variable>` is indeed case sensitive since it will be published to the `MOOSDB` and `MOOS` variables are case sensitive when registered for by a client. The `<value>` may or may not be case sensitive depending on whether or not a client registering for the variable regards the case. Considering again the helm `Iterate()` loop depicted in Figure 1, variable initializations are applied to the helm's information buffer prior to the very first helm iteration, but are posted to the `MOOSDB` at the end of the first helm iteration.

By default, an initialization will overwrite any prior value posted to the `MOOSDB`. There may be situations, however, where the user's desired effect is that the initialization only be applied if no other value has yet been written to the given `MOOS` variable. The syntax in this case would be:

```
initialize_ <variable> = <value>    // Deferring to prior posts if any
```

By using the "underscore" version of the `initialize` declaration, the helm will first register with the `MOOSDB` for the given variable, wait an iteration until it has had chance to receive mail from the `MOOSDB` on that variable, and only initialize the variable if nothing is otherwise known about that variable. (Note to the very discerning reader: Such an initialization also includes both an update to the helm's information buffer and a post to the `MOOSDB`. Posts to the `MOOSDB` by the helm, as part of a variable initialization, will indeed show up in the helm's incoming mailbox on the next iteration, but they are tagged in such a way as to be ignored by the helm. This is to ensure that they do not "collide" with posts made by other processes.)

1.3.2 Behavior Configuration Syntax

The bulk of the helm configuration is done with individual behavior parameter blocks which have the following form:

```

Behavior = <behavior-type>
{
    <parameter> = <value>
    ...
    <parameter> = <value>
}

```

The first line is a declaration of the behavior type. The keyword `Behavior` is not case sensitive, but the `<behavior-type>` is. This is followed by an open brace on a separate line. Each subsequent line sets a particular parameter of the behavior to a given value. The behavior configuration concludes with a close brace on a separate line. The issue of case sensitivity for the `<parameter>` and `<value>` entries is a matter determined by the individual behavior implementation.

As a convention (not enforced in any way) general behavior parameters, defined at the IvP Behavior superclass level, are grouped together and listed before parameters that apply to a specific behavior. For example, in the Alpha, the general behavior parameters are listed on lines 8-12 and 22-25, but the parameters specific to the waypoint behavior, `speed`, `radius`, and `points`, follow in a separate block. Generally it is not mandatory to provide a parameter-value pair for each parameter defined for a behavior, given that meaningful defaults are in place within the behavior implementation. Some parameters are indeed mandatory however. Documentation for the individual behavior should be consulted. Multiple instances of a behavior type are allowed, as in the Alpha example where there are two waypoint behaviors - one for traversing a set of points, and one for returning to a vehicle recovery point. Each behavior should have its own unique value provided in the `name` parameter.

1.3.3 Hierarchical Mode Declaration Syntax

Hierarchical Mode Declarations are covered in depth in Section 1.4, but the syntax is briefly discussed here. A behavior file contains a set of declaration blocks of the form:

```

Set <mode-variable-name> = <mode-value>
{
    <mode-variable-name> = <parent-value>
    <condition>
    . . .
    <condition>
} <else-value>

```

A tree will be formed where each node in the tree is described from the above type of declaration. The keyword `Set` is case insensitive. The `<mode-variable-name>`, `<parent-value>` and `<else-value>` are case sensitive. The `<condition>` entries are treated exactly as with the `condition` parameter for behaviors.

As indicated in Figure 1, the value of each mode variable is reset at the outset of the `Iterate()` loop, after the information buffer is updated with incoming mail. A mode variable is set by progressing through each declaration block, and determining whether the conditions are met. Thus the ordering of the declaration blocks is significant - the specification of parent should be made prior to that of a child. Examples are further discussion can be found below in Section 1.4.

1.4 Hierarchical Mode Declarations

Hierarchical mode declarations (HMDs) are an optional feature of the IvP Helm for organizing the behavior activations according to declared mission modes. Modes and sub-modes can be declared, in line with a mission planner's own concept of mission evolution, and behaviors can be associated with the declared modes. In more complex missions, it can facilitate mission planning (in terms of less time and better detection of human errors), and it can facilitate the understanding of exactly what is happening in the helm - during the mission execution and in post-analysis.

1.4.1 Background

A trend of unmanned vehicle usage can be characterized as being increasingly less of the shorter, scripted variety to be increasingly more of the longer, adaptive mission variety. A typical mission in our own lab five years ago would contain a certain set of tasks, typically waypoints and ultimately a rendezvous point for recovering the vehicle. Data acquired during deployment was off-loaded and analyzed later in the laboratory. What has changed? The simultaneous maturation of acoustic communications, on-board sensor processing, and longer vehicle battery life has dramatically changed the nature of mission configurations. The vehicle is expected to adapt to both the phenomena it senses and processes on board, as well as adapt its operation given field-control commands received via acoustic, radio or satellite communications. Multi-vehicle collaborative missions are also increasingly viable due to lower vehicle costs and mature acommms capabilities. In such cases a vehicle is not only adapting to sensed phenomena and field commands, but also to information from collaborating vehicles.

Our missions have evolved from having a finite set of fixed tasks to be composed instead of a set of modes, an initial mode when launched, an understanding of what brings us from one mode to another, and what behaviors are in play in each mode. Modes may be entered and exited any number of times, in exact sequences unknown at launch time, depending on what they sense and how they are commanded in the field.

1.4.2 Behavior Configuration *Without* Hierarchical Mode Declarations

Behaviors can be configured for a mission without the use of hierarchical mode declarations - support for HMDs is a relatively recent addition to the helm. HMDs are a tool for organizing which behaviors are idle or participating in which circumstances. Consider the alpha example mission, and the corresponding behavior file. By examination of the behavior file, and experimenting a bit with the viewer during simulation, the vehicle apparently is always in one of three modes - (a) idle, (b) surveying the waypoints, or (c) returning to the launch point. This is achieved by the `condition` parameters for the two behaviors. There are only two variables involved in the behavior conditions, `DEPLOY` and `RETURN`. If restricted to Boolean values, the below table confirms the observation that there are only three possible modes.

DEPLOY	RETURN	Mode
true	true	Returning
true	false	Surveying
false	true	Idle
false	false	Idle

Table 1: Possible modes implied by the condition parameters in the alpha mission.

There are a couple drawbacks with this however. First, the modes are to be inferred from the behavior conditions and this is not trivial in missions with larger behavior files. Mapping the behavior conditions to a mode is useful both in mission planning and mission monitoring. In the alpha mission, in order to understand at any given moment what mode the vehicle is in, the two variables need to be monitored, and the above table internalized. The second drawback is the increased likelihood of error, in the form of unintentionally being in two modes at the same time, or being in an undefined mode. For example, line 11 really should read `RETURN != true`, and not `RETURN = false`. Since there is no Boolean type for MOOS variables, this variable could be set to "False" and the condition as it reads on line 11 would not be satisfied, and the vehicle would be in the idle state, despite the fact that `DEPLOY` may be set to true. These problems are alleviated by the use of hierarchical mode declarations.

1.4.3 Syntax of Hierarchical Mode Declarations - The Bravo Mission

An example is provided showing of the use of hierarchical mode declarations by extending the Alpha mission described in Section ???. This example mission is dubbed the "Bravo" mission in the directory `s2.bravo` alongside the Alpha mission `s1.alpha` in the MOOS-IvP distribution (Section ??). It is also given fully in Listing 1 on the next page. The *implicit* modes of the Alpha mission, described in Table 1, are explicitly declared in the Bravo behavior file to form the following hierarchy:

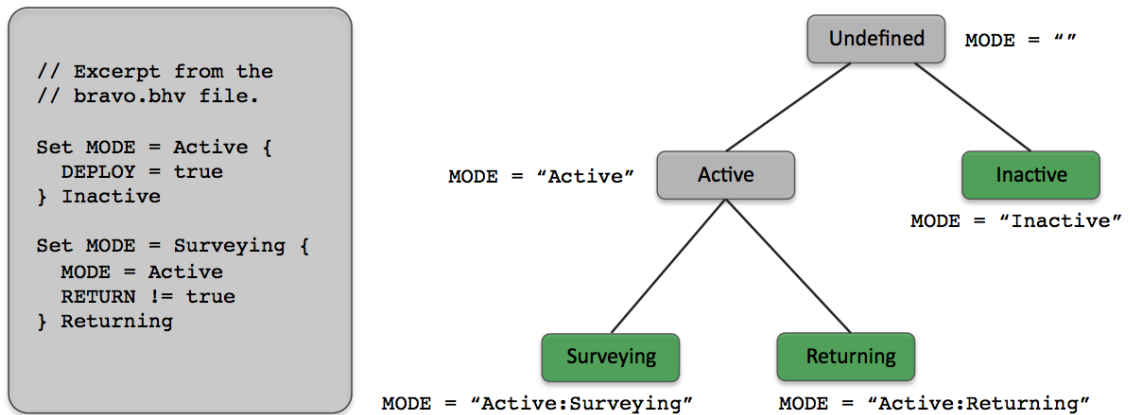


Figure 2: **Hierarchical modes for the Bravo mission:** The vehicle will always be in one of the modes represented by a leaf node. A behavior may be associated with any node in the tree. If a behavior is associated with an internal node, it is also associated with all its children.

The hierarchy in Figure 2 is formed by the mode declaration constructs on the left-hand side, taken as an excerpt from the `bravo.bhv` file. After the mode declarations are read when the helm is initially launched, the hierarchy remains static thereafter. The hierarchy is associated with a particular MOOS variable, in this case the variable `MODE`. Although the hierarchy remains static, the mode is re-evaluated at the outset of each helm iteration based on the conditions associated with nodes in the hierarchy. The mode evaluation is represented as a string in the variable `MODE`. As shown in Figure 2 the variable is the concatenation of the names of all the nodes. The mode evaluation begins sequentially through each of the blocks. At the outset the value of the variable `MODE` is reset to the empty string. After the first block in Figure 2 `MODE` will be set to either "Active" or "Inactive". When the second block is evaluated, the condition "`MODE=Active`" is evaluate based on how `MODE` was set in the first block. For this reason, mode declarations of children need to be listed after the declarations of parents in the behavior file.

Once the mode is evaluated, at the outset of the helm iteration, it is available for use in the conditions of the behaviors, as in lines 20 and 23 in Listing 1. Note the "==" relation in lines 18 and 36. This is a string-matching relation that matches when one side matches exactly one of the components in the other side's colon-separated list of strings. Thus "Active" == "Active:Returning", and "Returning" == "Active:Returning". This is to allow a behavior to be easily associated with an internal node regardless of its children. For example if a collision-avoidance behavior were to be added to this mission, it could be associated with the "Active" mode rather than explicitly naming all the sub-modes of the "Active" mode.

Listing 1.1: The Bravo Mission - Use of Hierarchical Mode Declarations.

```

1 initialize   DEPLOY = false
2 initialize   RETURN = false
3
4 //----- Declaration of Hierarchical Modes
5 set MODE = ACTIVE {
6   DEPLOY = true
7 } INACTIVE
8
9 set MODE = SURVEYING {
10  MODE = ACTIVE
11  RETURN != true
12 } RETURNING
13
14 //-----
15 Behavior = BHV_Waypoint
16 {
17   name      = waypt_survey
18   pwt       = 100
19   condition = MODE == SURVEYING
20   endflag   = RETURN = true
21   perpetual = true
22
23   lead = 8
24   lead_damper = 1
25   speed = 2.0 // meters per second
26   radius = 4.0
27   nm_radius = 10.0
28   points = 60,-40:60,-160:150,-160:180,-100:150,-40
29   repeat = 1
30 }
31
32 //-----

```

```

33 Behavior = BHV_Waypoint
34 {
35     name      = waypt_return
36     pwt       = 100
37     condition = MODE == RETURNING
38     perpetual = true
39     endflag   = RETURN = false
40     endflag   = DEPLOY = false
41
42     speed = 2.0
43     radius = 2.0
44     nm_radius = 8.0
45     point = 0,0
46 }

```

1.4.4 A More Complex Example of Hierarchical Mode Declarations

The Bravo example given above, while having the benefit of being a working example distributed with the codebase, is not complex. In this section a modestly complex, although fictional, hierarchy is provided to highlight some issues with the syntax. The hierarchy with the corresponding mode declarations are shown in Figure 3. The declarations are given in the order of layers of the tree ensuring that parents are declared prior to children. As with the Bravo example in Figure 2, the nodes that represent realizable modes are depicted in the darker (green) color.

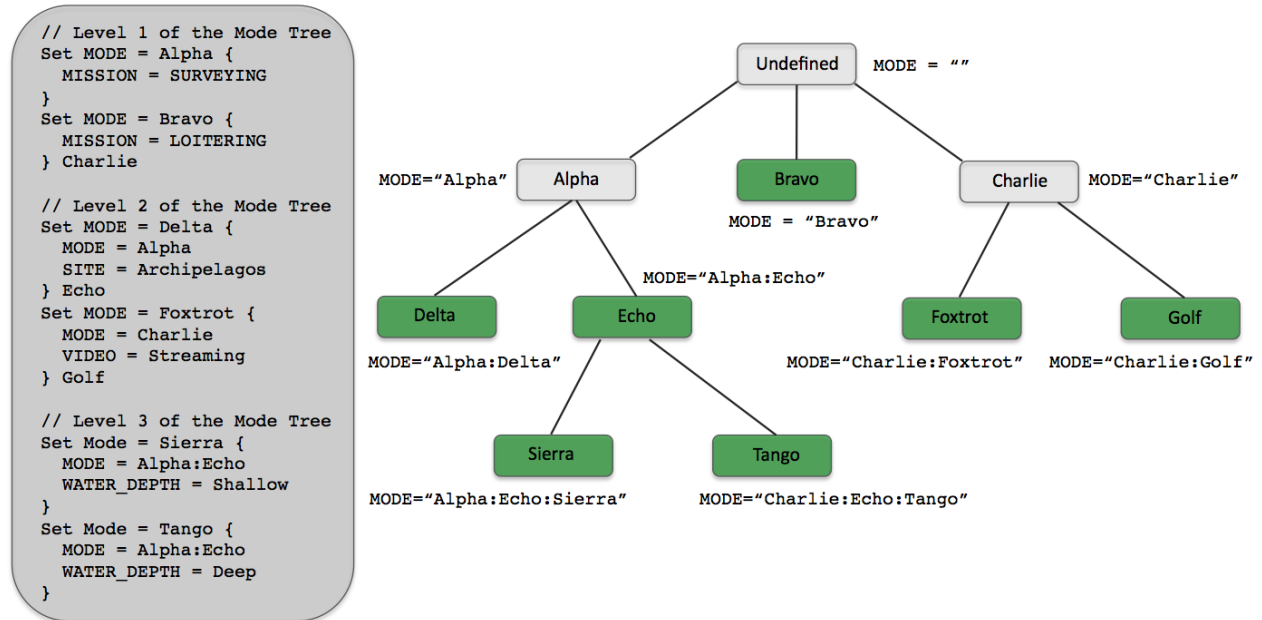


Figure 3: **Example Hierarchical Mode Declaration:** The hierarchy on the right is constructed from the set of mode declarations on the left (with fictional conditions). Darker nodes represent modes that are realizable through some combination of conditions.

The "Alpha" mode for example is not realizable since it has the children "Delta" and "Echo", with the latter being set as the <else-value> if the conditions of the former are not met. The "Bravo"

mode is realizable since it has no children. The "Echo" mode is realizable despite having children because the "Tango" mode is not the <else-value> of the "Sierra" mode declaration. For example, if the following three conditions hold, (a) "MISSION=SURVEYING", (b) "SITE!=Archipelagos", and (c) "WATER_DEPTH=Medium", then the value of the variable MODE would be set to "Alpha:Echo". Finally, note that the condition in the "Sierra" declaration, "MODE=Alpha:Echo", is specified fully, i.e., "MODE=Echo" would not achieve the desired result.

1.4.5 Monitoring the Mission Mode at Run Time

The mission mode can be monitored at run time in a couple ways. First, since the mode variable is posted as a MOOS variable, any MOOS scope tool will work, e.g., [uXMS](#), [uMS](#), [uHelmScope](#). Using [uHelmScope](#), the mission variable can be monitored as part of the basic [MOOSDB](#) scoping capability, but it is also displayed as part of the [uHelmScope](#) app, and in the AppCasting output of [pHelmIvP](#).

The [uHelmScope](#) tool also has a mode in which the entire mode hierarchy may be rendered - solely to provide a visual confirmation that the hierarchy specified with the mode declarations in the behavior file does in fact correspond to what the user intended. Currently there are no tools to automatically render the mode hierarchy in a manner like the right hand side of Figure 3. The [uHelmScope](#) output for the example in Figure 3 is shown in listing 2 below.

Listing 1.2: The mode hierarchy output from [uHelmScope](#) for the example in Figure 3.

```

1  ModeSet Hierarchy:
2  -----
3  Alpha
4      Delta
5      Echo
6          Sierra
7          Tango
8  Bravo
9  Charlie
10     Foxtrot
11     Golf
12  -----
13  CURENT MODE(S): Charlie:Foxtrot
14
15  Hit 'r' to resume outputs, or SPACEBAR for a single update

```

More on this feature of the [uHelmScope](#) can be found in the [uHelmScope](#) documentation. It's worth noting that poking the value of a mode variable will have no effect on the helm operation. The mission mode cannot be commanded directly. The mode variable is reset at the outset of the helm iteration, and the helm doesn't even register for mail on mode variables.

1.5 Behavior Participation in the IvP Helm

The primary work of the helm comes when the behaviors participate and do their thing, at each round of the helm [Iterate\(\)](#) loop. As depicted in Figure 1, once the mode has been re-evaluated taking into consideration newly received mail, it is time for the behaviors (well, some at least) to step up and do their thing.

1.5.1 Behavior Run Conditions

On any single iteration a behavior may participate by generating an objective function to influence the helm's output over its decision space. Not all behaviors participate in this regard, and the primary criteria for participation is whether or not it has met each of its "run conditions". These are the conditions laid out in the behavior file of the form:

```
condition = <logic-expression>
```

The <logic-expression> syntax is described in Appendix ???. Conditions are built from simple relational expressions, the comparison of MOOS variables to specified literal values, or the comparison of MOOS variables to one another. Conditions may also involve Boolean logic combinations of relation expressions. A behavior may base its conditions on any MOOS variable such as:

```
condition = (DEPLOY=true) and (STATION_KEEP != true)
```

A run condition may also be expressed in terms of a helm mode, as described in the next Section 1.5.2 such as:

```
condition = (MODE == LOITERING)
```

All MOOS variables involved in run condition expressions are automatically subscribed for by the helm to the MOOSDB.

1.5.2 Behavior Run Conditions and Mode Declarations

The use of hierarchical mode declarations potentially simplify the expressions used as run conditions. The conditions in practice could be limited to:

```
condition = <mode-variable> = <mode-value>, or  
condition = <mode-variable> == <mode-value>.
```

Conditions were used in this way with the Bravo mission in Listing 1, as an alternative to their usage in the Alpha mission example.

Note the use of the double-equals relation above. This relation is used for matching against the strings used to represent the hierarchical mode. The two strings match if the ordered components of one side are a subset of the ordered components of the other. Components are colon-separated. For example, using the illustrative hierarchy from Figure 3:

```
"Alpha:Echo:Sierra" == "Sierra"  
"Alpha:Echo:Sierra" == "Echo:Sierra"  
"Alpha:Echo:Sierra" == "Alpha"  
    "Sierra" == "Alpha:Echo:Sierra"  
    "Charlie:Foxtro" == "Charlie:Foxtro"  
  
"Alpha:Echo:Sierra" != "Alpha:Sierra"
```

1.5.3 Behavior Run States

On any given helm iteration a behavior may be in one of four states depicted in Figure 4:



Figure 4: **Behavior States:** A behavior may be in one of these four states at any given iteration of helm `Iterate()` loop. The state is determined by examination of MOOS variables stored locally in the helm’s information buffer.

- **Idle:** A behavior is `idle` if it is not `complete` and it has not met its run conditions as described above in Section ???. The helm will invoke an idle behavior’s `onIdleState()` function.
- **Running:** A behavior is `running` if it has met its run conditions and it is not `complete`. The helm will invoke a running behavior’s `onRunState()` function thereby giving the behavior an opportunity to contribute an objective function.
- **Active:** A behavior is `active` if it is running and it did indeed produce an objective function when prompted. There are a number of reasons why a running behavior may not be active. For example, a collision avoidance behavior where the object of the behavior is sufficiently far away.
- **Complete:** A behavior is `complete` when the behavior itself determines it to be complete. It is up to the behavior author to implement this, and some behaviors may never complete. The function `setComplete()` is defined generally at the behavior superclass level, for calling by a behavior author. This provides some standard steps to be taken upon completion, such as posting of `endflags`, described below in Section 1.5.4. Once a behavior is in the `complete` state, it remains in that state permanently. All behaviors have a `duration` parameter defined to allow it to be configured to time-out if desired. When a time-out occurs the behavior state will be set to `complete`.

1.5.4 Behavior Flags and Behavior Messages

Behaviors may post some number of messages, i.e., variable-value pairs, on any given iteration (see Figure 1). These message can be critical for coordinating behaviors with each other and to other MOOS processes. The can also be invaluable for monitoring and debugging behaviors configured for particular missions. To be more accurate, behaviors don’t post messages to the `MOOSDB`, they request the helm to post messages on its behalf. The helm collects these requests and publishes them to the `MOOSDB` at the end of the `Iterate()` loop. It also filters them for successive duplicates.

There is a standard method, configurable in the behavior file, for posting messages based on the run state of the behavior. These are referred to as behavior flags, and there are several types, `endflag`, `idleflag`, `runflag`, `activeflag`, `inactiveflag`, and `spawnflag`. The variable-value pairs representing each flag are set in the behavior file for the corresponding behavior.

- **endflag**: An **endflag** is posted once when or if the behavior enters the **complete** state. The variable-value pair representing the endflag is given in the **endflag** parameter in the behavior file. Multiple endflags may be configured for a behavior.
- **idleflag**: An **idleflag** is posted by the helm when the behavior enters the **idle** state. The variable-value pair representing the idleflag is given in the **idleflag** parameter in the behavior file. Multiple idleflags may be configured for a behavior.
- **runflag**: A **runflag** is posted by the helm when the behavior enters the **running** state from the **idle** state. A **runflag** is posted exactly when an **idleflag** is not. The variable-value pair representing the runflag is given in the **runflag** parameter in the behavior file. Multiple runflags may be configured for a behavior.
- **activeflag**: An **activeflag** is posted by the helm when the behavior enters the **active** state. The variable-value pair representing the activeflag is given in the **activeflag** parameter in the behavior file. Multiple activeflags may be configured for a behavior.
- **inactiveflag**: An **inactiveflag** is posted by the helm when the behavior enters a state that is not the **active** state. The variable-value pair representing the inactiveflag is given in the **inactiveflag** parameter in the behavior file. Multiple inactiveflags may be configured for a behavior.
- **spawnflag**: An **spawnflag** is posted by the helm when templated behavior is spawned. Examples include the collision and obstacle avoidance behaviors. The variable-value pair representing the spawnflag is given in the **spawnflag** parameter in the behavior file. Multiple spawnflags may be configured for a behavior.

A **runflag** is meant to "complement" an **idleflag**, by posting exactly when the other one does not. Similarly with the **inactiveflag** and **activeflag**. The situation is shown in Figure 5:



Figure 5: **Behavior Flags**: The four behavior flags **idleflag**, **runflag**, **activeflag**, and **inactiveflag** are posted depending on the behavior state and can be considered complementary in the manner indicated.

Behavior authors may implement their behaviors to post other messages as they see fit. For example the waypoint behavior used in the Alpha example also published the variable **WPT_STAT** with a status message similar to "vname=alpha,index=0,dist=124,eta=62" indicating the name of the vehicle, the index of the next point in the list of waypoints, the distance to that waypoint, and the estimated

time of arrival, in seconds. (You might want to re-run the Alpha mission with `uXMS` scoping on this variable to watch it change as the mission unfolds.)

1.5.5 Monitoring Behavior Run States and Messages During Mission Execution

The run states for each behavior, are wrapped up on each iteration by the helm into a single string and published in the variable `IVPHELM_SUMMARY`. This variable is subscribed for by the `uHelmScope` tool and behavior states are parsed from this variable and summarized in the main output of `uHelmScope`, as in the below excerpt:

```
12 Behaviors Active: ----- (1)
13   waypt_survey (13.0) (pwt=100.00) (pcs=1227) (cpu=0.01) (upd=0/0)
14 Behaviors Running: ----- (0)
15 Behaviors Idle: ----- (1)
16   waypt_return (22.8)
17 Behaviors Completed: ----- (0)
```

Behaviors are grouped into the four possible states, with a summary line for each state, e.g., lines 12, 14, 15, 17, containing the number of behaviors in that state in parentheses at the end of the line. Each behavior configured for the helm shows up on a dedicated line in the appropriate group, e.g., lines 13 and 16. In these lines immediately following the behavior name, the number of seconds is displayed in parentheses indicating how long the behavior has been in that state.

1.6 Behavior Reconciliation in the IvP Helm - Multi-Objective Optimization

1.6.1 IvP Functions

IvP functions are produced by behaviors to influence the decision produced by the helm on the current iteration (see Figure 1). The decision is typically comprised of the desired `heading`, `speed`, and `depth` but the helm decision space could be comprised of other variables. Some points about IvP functions:

- IvP functions are piecewise linearly defined. Each piece is defined by an interval over some subset of the decision space, and there is a linear function associated with each piece (see Figure 7).
- IvP functions are an *approximation* of an underlying function. The linear function for a single piece is the best linear approximation of the underlying function for the portion of the domain covered by that piece.
- IvP domains are discrete with an upper and lower bound for each variable, so an IvP function *may* achieve zero-error in approximating an underlying function by associating a piece with each point in the domain. Behaviors seldom need to do so in practice however.
- The IvP function construct and IvP solver are generalizable to N dimensions.
- The pieces in IvP functions need not be uniform size or shape. More pieces can be dedicated to parts of the domain that are harder to approximate with linear functions.

- IvP functions need only be defined over a subset of the domain. Behaviors are not affected if the helm is configured for additional variables that a behavior may not care about. Behaviors that produce functions solely over vehicle `depth` are perfectly ok.

How are IvP functions built? The IvP Build Toolbox is a set of tools for creating IvP functions based on any underlying function defined over an IvP Domain. Many, if not all of the behaviors in this document make use of this toolbox, and authors of new behaviors have this at their disposal. A primary component of writing a new behavior is the development of the "underlying function", the function approximated by an IvP function with the help of the toolbox. The underlying function represents the relationship between a candidate helm decision and the expected utility with respect to the behavior's objectives. The IvP Toolbox is not covered in detail in this document, but an overview is given below.

1.6.2 The IvP Build Toolbox

The IvP Toolbox is a set of tools (a C++ library) for building IvP functions. It is typically utilized by behavior authors in a sequence of library calls within a behavior's (C++) implementation. There are two sets of tools - the *Reflector* tools for building IvP functions in N dimensions, and the *ZAIC* tools for building IvP functions in one dimension as a special case. The Reflector tools work by making available a function to be approximated by an IvP function. The tools simply need this function for sampling. Consider the Gaussian function rendered below in Figure 6:

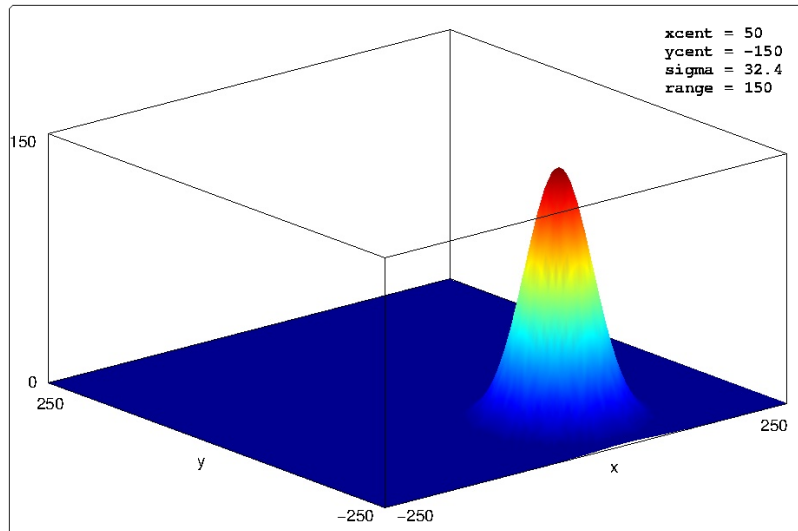


Figure 6: A rendering of the function $f(x, y) = Ae^{-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}}$ where $A = \text{range} = 150$, $\sigma = \text{sigma} = 32.4$, $x_0 = \text{xcent} = 50$, $y_0 = \text{ycent} = -150$. The domain here for x and y ranges from -250 to 250 .

The 'x' and 'y' variables, each with a range of $[-250, 250]$, are discrete, taking on integer values. The domain therefore contains $501^2 = 251,001$ points, or possible decisions. The IvP Build Toolbox can generate an IvP function approximating this function over this domain by using a uniform piece size, as rendered in Figure 7. The only difference in these four piecewise function is the number

and size of the piece. More pieces (Figure 7 (a)) results in a more accurate approximation of the underlying function, but takes longer to generate and creates further work for the IvP solver when the functions are combined. IvP functions need not use uniformly sized pieces.

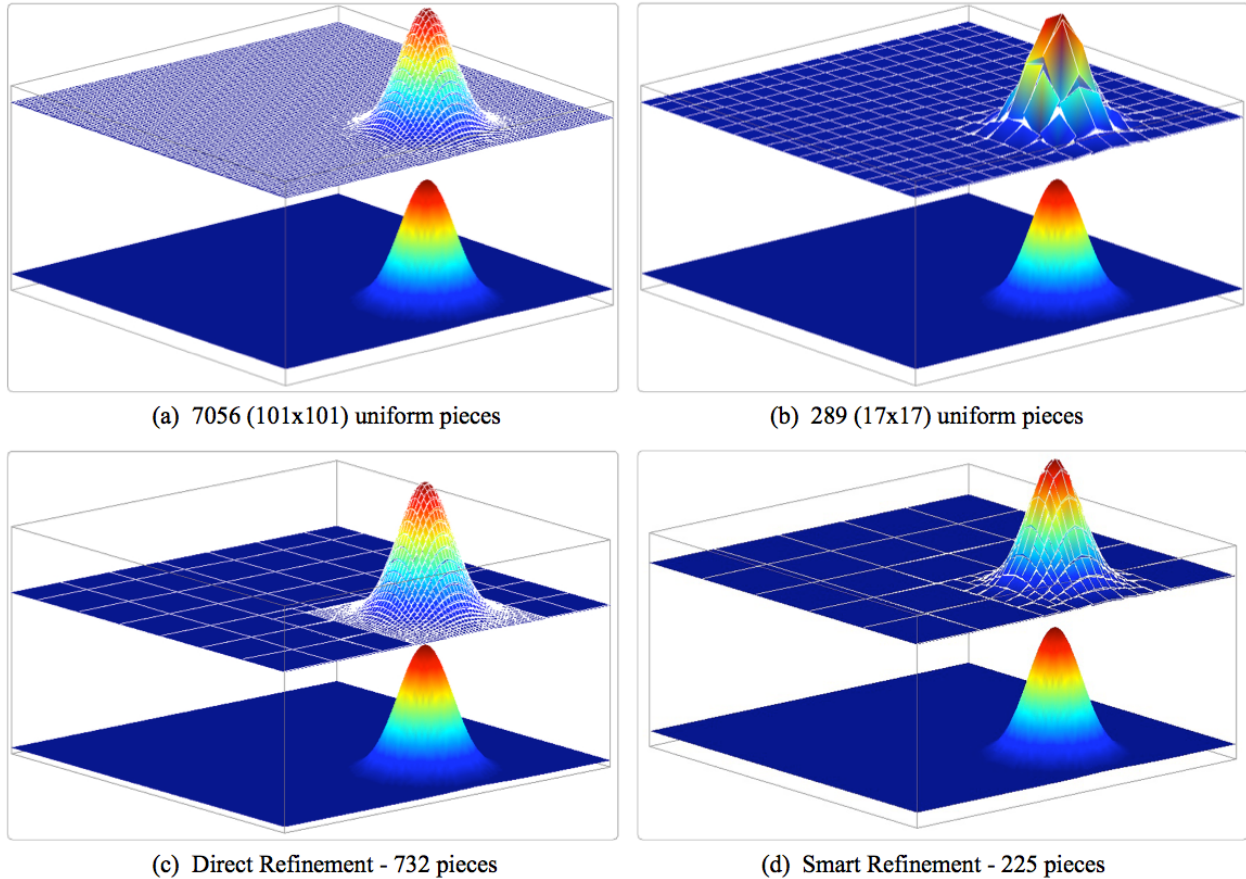


Figure 7: **A rendering of four different IvP functions approximating the same underlying function:** The function in (a) uses a uniform distribution of 7056 pieces. The function in (b) uses a uniform distribution of 1024 pieces. The function in (c) was created by first building a uniform distribution of 49 pieces and then focusing the refinement on a sub-domain of the function. This is called directed-refinement in the IvP Build toolbox. The function in (d) was created by first building a uniform function of 25 pieces and repeatedly refining the function based on which pieces were noted to have a poor fit to the underlying function. This is termed smart-refinement in the IvP Build toolbox.

By using the *directed refinement* option in the IvP Build Toolbox, an initially uniform IvP function can be further refined with more pieces over a sub-domain directed by the caller, with smaller uniform pieces of the caller's choosing. This is described more fully in the documentation for the IvP Build Toolbox. Using this tool requires the caller to have some idea where, in the sub-domain, further refinement is needed or desired. Often a behavior author indeed has this insight. For example, if one of the domain variables is vehicle heading, it may be good to have a fine refinement in the neighborhood of heading values close to the vehicle's current heading.

In other situations, insight into where further refinement is needed may not be available to the caller.

In these cases, using the *smart refinement* option of the IvP Build Toolbox, an initially uniform IvP function may be further refined by asking the toolbox to automatically "grade" the pieces as they are being created. The grading is in terms of how accurate the linear fit is between the piece's linear function and the underlying function over the sub-domain for that piece. A priority queue is maintained based on the grades, and pieces where poor fits are noted, are automatically refined further, up to a maximum piece limit chosen by the caller. This is described more fully in the documentation for the IvP Build Toolbox

The Reflector tools work similarly in N dimensions and on multi-modal functions. The only requirement for using the Reflector tool is to provide it with access to the underlying function. Since the tool repetitively samples this function, a central challenge to the user of the toolbox is to develop a fast implementation of the function. In terms of the time consumed in generating IvP functions with the Reflector tool, the sampling of the underlying function is typically the long pole in the tent.

1.6.3 The IvP Solver and Behavior Priority Weights

The IvP Solver collects a set of weighted IvP functions produced by each of the behaviors and finds a point in the decision space that optimizes the weighted combination. If each IvP objective function is represented by $f_i(\vec{x})$, and the weight of each function is given by w_i , the solution to a problem with k functions is given by:

$$\vec{x}^* = \underset{\vec{x}}{\operatorname{argmax}} \sum_{i=0}^{k-1} w_i f_i(\vec{x})$$

The algorithm is described in detail in [Benjamin 2004], but is summarized in the following few points.

- *The search tree:* The structure of the search algorithm is branch-and-bound. The search tree is comprised of an IvP function at each layer, and the nodes at each layer are comprised of the individual pieces from the function at that layer. A leaf node represents a single piece from each function. A node in the tree is realizable if the piece from that node and its ancestors intersect, i.e., share common points in the decision space.
- *Global optimality:* Each point in the decision space is in exactly one piece in each IvP function and is thus in exactly one leaf node of the search tree. If the search tree is expanded fully, or pruned properly (only when the pruned out sub-tree does not contain the optimal solution), then the search is guaranteed to produce the globally optimal solution. The search algorithm employed by the IvP solver does indeed start with a fully expanded tree, and utilizes proper pruning to guarantee global optimality. The algorithm does allow for a parameter for guaranteed limited back-off from the global optimality - a quicker solution with a guarantee of being within a fixed percent of global optima. This option is not exposed to the IvP Helm which always finds the global optimum.
- *Initial solution:* A key factor of an effective branch-and-bound algorithm is seeding the search with a decent initial solution. In the IvP Helm, the initial solution used is the solution (typically

heading, speed, depth) generated on the previous helm iteration. Upon casual observation this appears to provide a speed-up by about a factor of two.

In cases where there is a "tie" between optimal decisions, the solution generated by the solver is non-deterministic. This is mitigated somewhat by the fact that the solution is seeded with the output of the previous iteration as discussed above.

1.6.4 Monitoring the IvP Solver During Mission Execution

The performance of the solver can be monitored with the `uHelmScope` tool. The output shown below is an excerpt from an example mission. On line 5, the total time needed to solve the multi-objective optimization problem is given in seconds, and the max time need for all recorded loops is given in parentheses. It is zero here since there is only one objective function in this example. On line 6 is the total time for creating the IvP functions in all behaviors, with the max across all iterations in parentheses. On line 7 is the total loop time - the sum of the previous two lines. Active behaviors display useful information regarding the IvP solver. For example, on line 13, the Survey waypoint behavior had a priority weight of 100 and generated 1,227 pieces, taking 0.01 seconds of CPU time to create.

Listing 1.3: Example `uHelmScope` output containing information about the IvP solver.

```

1  ===== uHelmScope Report ===== DRIVE (17)
2  Helm Iteration: 66      (hz=0.38)(5) (hz=0.35)(66) (hz=0.56)(max)
3  IvP functions: 1
4  Mode(s):      Surveying
5  SolveTime:    0.00      (max=0.00)
6  CreateTime:   0.02      (max=0.02)
7  LoopTime:     0.02      (max=0.02)
8  Halted:       false    (0 warnings)
9  Helm Decision: [speed,0,4,21] [course,0,359,360]
10 speed = 3.00
11 course = 177.00
12 Behaviors Active: ----- (1)
13   waypt_survey (13.0) (pwt=100.00) (pcs=1227) (cpu=0.01) (upd=0/0)
14 Behaviors Running: ----- (0)
15 Behaviors Idle: ----- (1)
16   waypt_return (22.8)
17 Behaviors Completed: ----- (0)
18
```

The solver can be additionally monitored and analyzed through the two MOOS variables `LOOP_CPU` and `CREATE_CPU` published on each helm iteration. The former indicates the system wall time for building each IvP function and solving the multi-objective optimization problem, and the latter indicates just the time to create the IvP functions.

References

- Paolo Pirjanian, Multiple Objective Action Selection and Behavior Fusion, PhD Thesis, Aalborg University, 1998.
- Rodney A. Brooks, A Robust Layered Control System for a Mobile Robot, IEEE Journal of Robotics and Automation, April 1986.

- George B. Dantzig, *Programming in a Linear Structure*, Comptroller, United States Air Force, February, 1948.
- Julio K. Rosenblatt, *DAMN: A Distributed Architecture for Mobile Navigation*, Journal of Experimental and Theoretical Artificial Intelligence, Volume 9, Number 2, 1997.
- Michael R. Benjamin, *The Interval Programming Model for Multi-Objective Decision Making*, Computer Science and Artificial Intelligence Laboratory, AIM-2004-021, MIT, Cambridge, MA.

Properties of Helm Behaviors

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_bhv_props

1	Properties of Helm Behaviors	1
1.1	Brief Overview	1
1.2	Parameters Common to All IvP Behaviors	2
1.3	Overloading the <code>setParam()</code> Function in New Behaviors	9
1.4	Behavior Functions Invoked by the Helm	10
1.5	Local Behavior Utility Functions	13
1.6	Overloading the <code>onRunState()</code> and <code>onIdleState()</code> Functions	16
1.7	Dynamic Behavior Spawning	17

Contents

1 Properties of Helm Behaviors

The objective of this section is to describe properties common to all IvP Helm behaviors, describe how to overload standard functions for 3rd party behaviors, and to provide a detailed simple example of a behavior.

1.1 Brief Overview

Behaviors are implemented as C++ classes with the helm having one or more instances at runtime, each with a unique descriptor. The properties and implemented functions of a particular behavior are partly derived from the `IvPBehavior` superclass, shown in Figure 1. The is-a relationship of a derived class provides a form of code re-use as well as a common interface for constructing mission files with behaviors.

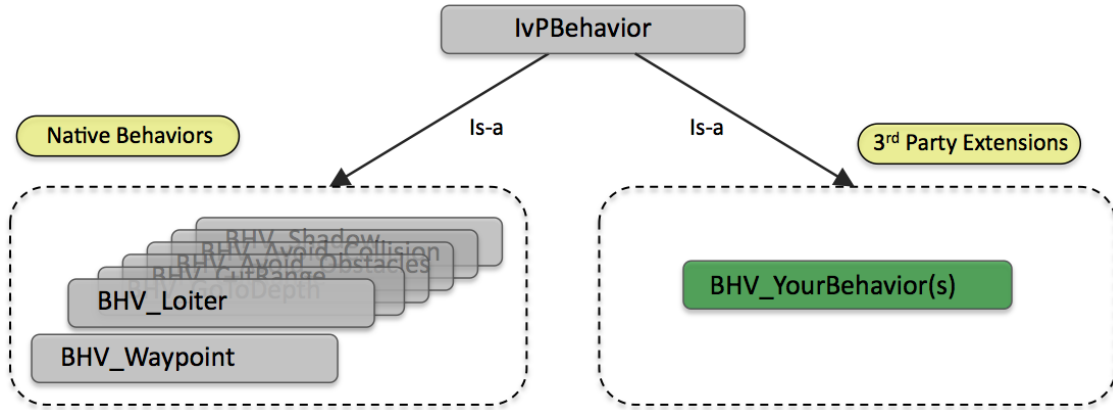


Figure 1: **Behavior inheritance**: Behaviors are derived from the `IvPBehavior` superclass. The native behaviors are the behaviors distributed with the helm. New behaviors also need to be a subclass of the `IvPBehavior` class to work with the helm. Certain virtual functions invoked by the helm may be optionally but typically overloaded in all new behaviors. Other private functions may be invoked within a behavior function as a way of facilitating common tasks involved in implementing a behavior.

The `IvPBehavior` class provides five virtual functions which are typically overloaded in a particular behavior implementation:

- The `setParam()` function: parameter-value pairs are handled to configure a behavior's unique properties distinct from its superclass.
- The `onRunState()` function: the meat of a behavior implementation, performed when the behavior has met its conditions for running, with the output being an objective function and a possibly empty set of variable-value pairs for posting to the `MOOSDB`.
- The `onIdleState()` function: what the behavior does when it has not met its run conditions. It may involve updating internal state history, generation of variable-value pairs for posting to the `MOOSDB`, or absolutely nothing at all.
- The `onIdleToRunState()` function: invoked once by the helm upon transitioning from the idle to running state (compared to the `onRunState()` function which is invoked on *each* helm iteration where the behavior has met its conditions).
- The `onRunToIdleState()` function: invoked once by the helm upon transitioning from the running to idle state (compared to the `onIdleState()` function which is invoked on *each* helm iteration where the behavior has not met its conditions).

This section discusses the properties of the `IvPBehavior` superclass that an author of a third-party behavior needs to be aware of in implementing new behaviors. It is also relevant material for users of the native behaviors as it details general properties.

1.2 Parameters Common to All IvP Behaviors

A behavior has a standard set of parameters defined at the `IvPBehavior` level as well as unique parameters defined at the subclass level. By configuring a behavior during mission planning, the setting of parameters is the primary venue for affecting the overall autonomy behavior in a vehicle.

Parameters are set in the behavior file, but can also be dynamically altered once the mission has commenced. A parameter is set with a single line of the form:

```
parameter = value
```

The left-hand side, the parameter component, is case insensitive, while the value component is typically case sensitive. This was discussed in depth in Section ???. In this section, the parameters defined at the superclass level and available to all behaviors are exhaustively listed and discussed. Each behavior typically augments these parameters with new ones unique to the behavior, and in the next section the issue of implementing new parameters by overloading the `setParam()` function is addressed.

1.2.1 A Summary of the Full Set of General Behavior Parameters

The following parameters are defined for all behaviors at the superclass level. They are listed here for reference - certain related aspects are discussed in further detail in other sections.

Listing 1.1: Configuration Parameters Common to All Behaviors.

- activeflag:** This parameter specifies a variable and a value to be posted when the behavior is in the *active* state. See the Section ?? for more on run states. It is only posted if the behavior, on the previous helm iteration, was not in the *active* state. It is an equal-separated pair such as `TRANSITING=true`. More than one flag may be provided. These can be used to satisfy or block the conditions of other behaviors.
- condition:** This parameter specifies a condition that must be met for the behavior to be active. Conditions are checked for each behavior at the beginning of each control loop iteration. Conditions are based on current MOOS variables, such as `STATE = normal` or `(K < 4)`. More than one condition may be provided, as a convenience, treated collectively as a single conjunctive condition. The helm automatically subscribes for any condition variables. See Section ?? for more detail on run conditions.
- configflag:** More info TBD
- comms_policy:** This parameter governs inter-vehicle messaging for messages originating within a behavior. More info TBD.
- duration:** The time in seconds that the behavior will remain running before declaring completion. If no duration value is provided, the behavior will never time-out. The clock starts ticking once the behavior satisfies its run conditions (becoming non-idle) the first time. *Should the behavior switch between running and idle states, the clock keeps ticking even during the idle periods.* See Section 1.2.6 for more detail.
- duration_idle_decay:** If this parameter is false the duration clock is paused when the vehicle is in the *idle* state. The default value is true. See Section 1.2.6 for more detail.

- duration_reset:** This parameter takes a variable-pair such as `MY_RESET=true`. If the `duration` parameter is set, the duration clock is reset when the variable is posted to the `MOOSDB` with the specified value. Each time such a post is noted, the duration clock is reset. See Section 1.2.6 for more detail.
- duration_status:** If the `duration` parameter is set, the remaining duration time, in seconds, can be posted by naming a `duration_status` variable. This variable will be update/posted only when the behavior is in the running state. See Section 1.2.6 for more detail.
- endflag:** This parameter specifies a variable and a value to be posted when the behavior has set the completed state variable to be true. The circumstances causing completion are unique to the individual behavior. However, if the behavior has a `duration` specified, the completed flag is set to true when the duration is exceeded. The value of this parameter is a equal-separated pair such as `ARRIVED_HOME=true`. Once the completed flag is set to true for a behavior, it remains inactive thereafter, regardless of future events, barring a complete helm restart.
- idleflag:** This parameter specifies a variable and a value to be posted when the behavior is in the *idle* state. See the Section ?? for more on run states. It is only posted if the behavior, on the previous helm iteration, was not in the *idle* state. It is an equal-separated pair such as `WAITING=true`. More than one flag may be provided. These can be used to satisfy or block the conditions of other behaviors.
- inactiveflag:** This parameter specifies a variable and a value to be posted when the behavior is *not* in the *active* state. See the Section ?? for more on run states. It is only posted if the behavior, on the previous helm iteration, was in the *active* state. It is a equal-separated pair such as `OUT_OF_RANGE=true`. More than one flag may be specified by the user in the behavior configuration. These can be used to satisfy or block the conditions of other behaviors.
- name:** The name of the behavior - should be unique between all behaviors. Due to the implementation of behavior templating, spawned behavior take on a new name by concatenating on the end of a base name. For this reason all configured behavior names could be regarded as base names. The check for uniqueness includes hypothetical extension. Thus the names `loiter` and `loiter.two` are not regarded as safe since the first could potentially grow into the latter with the concatenation of `.two`. Logging and output sent to the helm console during operation will organize information by the behavior name.
- nostarve:** The `nostarve` parameter allows a behavior to assert a maximum staleness for one or more MOOS variables, i.e., the time since the variable was last updated. The syntax for this parameter is a comma-separated pair "`variable, ..., variable, value`", where last component in the list is the time value given in seconds. See Section 1.2.9 for more detail.

- perpetual:** Setting the **perpetual** parameter to true allows the behavior to continue to run even after it has completed and posted its end flags. The parameter value is not case sensitive and the only two legal values are true and false. See Section 1.2.7 for more detail.
- post_mapping:** This parameter takes a comma-separated pair such as **WPT_STAT, WAYPT_STATUS** where the left-hand value is a variable normally posted by the behavior, and the right-hand value is an alternative variable name to be used. There is no error-checking to ensure that the left-hand value names a variable actually posted by the behavior. Transitive relationships are not respected. For example, if the two re-mappings are declared, **FOO, BAR**, and **BAR, CAR**, **FOO** will be posted as **BAR**, not **CAR**. To disable the normal posting of a variable **FOO**, use **post_mapping = FOO, SILENT**.
- priority:** The priority weight of the produced objective function. The default value is 100. A behavior may also be implemented to determine its own priority weight depending on information about the world.
- runflag:** This parameter specifies a variable and a value to be posted when the behavior has met all its conditions for being in the *running* state. It is only posted if the behavior, on the previous helm iteration, was not in the *running* state. It is an equal-separated pair such as **TRANSITING=true**. More than one flag may be provided. These can be used to satisfy or block the conditions of other behaviors.
- runxflag:** More info TBD
- spawnflag:** More info TBD
- spawnxflag:** More info TBD
- templating:** The **templating** parameter may be used to turn a behavior specification into a template for spawning new behaviors dynamically after the helm has been launched. Instantiation requests are received via the **updates** parameter described in Section 1.7.
- updates:** This parameter specifies a variable from which updates to behavior configuration parameters are read from after the behavior has been initially instantiated and configured at the helm startup time. Any parameter and value pair that would have been legal at startup time is legal at run-time. The syntax for this string is a #-separated list of parameter-value pairs: "param=value # param=value # ... # param=value". This is one of the primary hooks to the helm for mission control - the other being the behavior conditions described above. See Section 1.2.5 for more detail.

1.2.2 The **name** Parameter

The **name** parameter is a mandatory parameter with no default value that assigns a name to the behavior instance. The name must be unique. The behavior name is case sensitive, so having the names "return_home" and "Return_Home" is acceptable (although not advisable).

Due to the capability of *behavior templating* (Section 1.7) the policy of behavior uniqueness was expanded to include pairs of names where one name matches the beginning component of another.

For example "return" and "return_home" are not allowed. This is due to way template spawning is implemented. The spawning process begins by setting the name of the newly spawned behavior. This is achieved by appending to the *base name* declared for the behavior template. By allowing, for example, "return" and "return_home", it leaves open the possibility that the first behavior may spawn an instance with a name clash if it were spawned with a suffix directive of "_home".

Behavior names have no relation to the behavior class name. For example an instance of the Waypoint behavior may be called "station-keep" (although not advisable). Furthermore, multiple instances of the same behavior may certainly co-exist so long as each instance has a unique name. If the helm is configured with one or more behaviors with non-unique names, the helm will come up in the "MALCONFIG" state. The terminal or appcating output would look something like:

```
=====
pHelmIvP alpha                                1/0(411)
=====
Configuration Warnings: 1
[1 of 1]: Duplicate behavior name found: waypt_return

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!                                     !!
!!  The helm is in the MALCONFIG state due to  !!
!!  unresolved configuration warnings.          !!
!!                                     !!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

1.2.3 The **priority** Parameter

The **priority** parameter is a optional parameter with a default value of 100. It assigns a priority weight to be applied to an objective function produced by the behavior. Many behaviors interpret this priority to establish a range of priorities depending on the further evaluation of the situation by the behavior. For example, the collision avoidance behavior will treat the configured priority to be the maximum priority (when a contact is close) and will degrade the priority accordingly as the contact range is greater.

The range of acceptable values is from zero to infinity. If a negative value is given for a behavior the helm will come up in the "MALCONFIG" state. The terminal or appcating output would look something like:

```
=====
pHelmIvP alpha                                1/0(21)
=====
Configuration Warnings: 1
[1 of 1]: alpha.bhv: Line 61: pwt    = -1

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!                                     !!
!!  The helm is in the MALCONFIG state due to  !!
!!  unresolved configuration warnings.          !!
!!                                     !!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

1.2.4 The `post_mapping` Parameter

The `post_mapping` parameter is an optional parameter with no default value. It allows the user to configure a behavior to change the name of the variables otherwise posted by the behavior. For example, the Waypoint behavior by default publishes a variable `WPT_INDEX` indicating the index of the next point on its path. If there are multiple waypoint behaviors, or if you just don't like this variable, it may be changed for example to `SURVEY_INDEX` with the following configuration:

```
post_mapping=WPT_INDEX,SURVEY_INDEX
```

1.2.5 Altering Parameters Dynamically with the `updates` Parameter

The parameters of a behavior can be made to allow dynamic modifications - after the helm has been launched and executing the initial mission in the behavior file. The modifications come in a single MOOS variable specified by the parameter `updates`. For example, consider the simple waypoint behavior configuration below in Listing 2. The return point is the (0,0) point in local coordinates, and return speed is 2.0 meters/second. When the conditions are met, this is what will be executed.

Listing 1.2: An example behavior configuration using the `updates` parameter.

```
1 Behavior = BHV_Waypoint
2 {
3     name      = WAYPT_RETURN
4     priority  = 100
5     speed     = 2.0
6     radius    = 8.0
7     points    = 0,0
8     updates   = RETURN_UPDATES
9     condition = RETURN = true
10    condition = DEPLOY = true
11 }
```

If, during the course of events, a different return point or speed is desired, this behavior can be altered dynamically by writing to the variable specified by the `updates` parameter, in this case the variable `RETURN_UPDATES` (line 8 in Listing 2). The syntax for this variable is of the form:

```
parameter = value # parameter = value # ... # parameter = value
```

White space is ignored. The '#' character is treated as special for parsing the line into separate parameter-value pairs. It cannot be part of a parameter component or value component. For example, the return point and speed for this behavior could be altered by any other MOOS process that writes to the MOOS variable:

```
RETURN_UPDATES = "points=50,50 # speed = 1.5"
```

Each parameter-value pair is passed to the same parameter setting routines used by the behavior on initialization. The only difference is that an erroneous parameter-value pair will simply be ignored as opposed to halting the helm as done on startup. If a faulty parameter-value pair is encountered, a warning will be written to the variable `BHV_WARNING`. For example:

```
BHV_WARNING = "Faulty update for behavior: WAYPT_RETURN. Bad parameter(s): speed."
```

Note that a check for parameter updates is made at the outset of helm iteration loop for a behavior (Figure 1) with the call `checkUpdates()`. Any updates received by the helm on the current iteration will be applied prior to behavior execution and in effect for the current iteration.

1.2.6 Limiting Behavior Duration with the `duration` Parameter

The `duration` parameter specifies a time period in seconds before a behavior times out and permanently enters the completed state (Figure 2). If left unspecified, there is no time limit to the behavior. By default, the duration clock begins ticking as soon as the helm is in drive. The duration clock remains ticking when or if the behavior subsequently enters the idle state. It even remains ticking if the helm temporarily parks. When a timeout occurs, end flags are posted. The behavior can be configured to post the time remaining before a timeout with the `duration.status` parameter. The forms for each are:

```
duration          = value (positive numerical)
duration_status = value (variable name)
```

Note that the duration status variable will only be published/updated when the behavior is in the running state. The duration status is rounded to the nearest integer until less than ten seconds remain, after which the time is posted out to two decimal places. The behavior can be configured to have the duration clock pause when it is in the idle state with the following:

```
duration_idle_decay = false // The default is true
```

Configured in the above manner, a behavior's duration clock will remain paused until it's conditions are met. The behavior may also be configured to allow for the duration clock to be reset upon the writing of a MOOS variable with a particular value. For example:

```
duration_reset = BRAVO_TIMER_RESET=true
```

The behavior checks for and notes that the variable-value pair holds true and the duration clock is then reset to the original duration value. The behavior also marks the time at which the variable-value pair was noted to have held true. Thus there is no need to "un-set" the variable-value pair, e.g., setting `BRAVO_TIMER_RESET=false`, to allow the duration clock to resume its count-down.

1.2.7 The `perpetual` Parameter

When a behavior enters the *completed* state, it by default remains in that state with no chance to change. When the `perpetual` parameter is set to true, a behavior that is declared to be complete does not actually enter the complete state but performs all the other activity normally associated with completion, such as the posting of end flags. See Section ?? for more detail on posting flags to the `MOOSDB` from the helm. The default value for `perpetual` is false. The form for this parameter is:

```
perpetual = value
```

The value component is case insensitive, and the only legal values are either true or false. A behavior using the `duration` parameter with `perpetual` set to true will post its end flags upon time out, but will reset its clock and begin the count-down once more the next time its run conditions are met, i.e., enters the running state. Typically when a behavior is used in this way, it also posts an endflag that would put itself in the idle state, waiting for an external event.

1.2.8 The `post_mapping` Parameter

This parameter takes a comma-separated pair where the left-hand value is a variable normally posted by the behavior, and the right-hand value is an alternative variable name to be used. For example, the Waypoint behavior normally posts the variable `WPT_STAT` when it is running. This can be changed with:

```
post_mapping = WPT_STAT, WAYPT_STATUS
```

Note there is no error-checking to ensure that the left-hand value names a variable actually posted by the behavior. Transitive relationships are not respected. For example, if the two re-mappings are declared, `FOO,BAR`, and `BAR,CAR`, `FOO` will be posted as `BAR`, not `CAR`. To disable the normal posting of a variable `FOO`, use:

```
post_mapping = FOO, SILENT
```

1.2.9 Detection of Stale Variables with the `nostarve` Parameter

A behavior utilizing a variable generated by a MOOS process outside the helm, may require the variable to be sufficiently up-to-date. The staleness of a variable is the time since it was last written to by any process. The `nostarve` parameter allows the mission writer to set a staleness threshold. The form for this parameters is:

```
nostarve = variable_1, ...,variable_n, duration
```

The value of this parameter is a comma-separated list such as `nostarve = NAV_X, NAV_Y, 5.0`. The variable components name MOOS variables and the duration component, the last entry in the list, represents the tolerated staleness in seconds. If staleness is detected, a behavior failure condition is triggered which will trigger the helm to post all-stop values and relinquish to manual control.

1.3 Overloading the `setParam()` Function in New Behaviors

The `setParam()` function is a virtual function defined in the `IvPBehavior` class, with parameters implemented in the superclass (Section 1.2) handled in the superclass version of this function:

```
bool IvPBehavior::setParam(string parameter, string value);
```

The `setParam()` function should return true if the parameter is recognized and the value is in an acceptable form. In the rare case that a new behavior has no additional parameters, leaving this function undefined in the subclass is appropriate. The example below in Listing 3 gives an example for a fictional behavior `BHV_YourBehavior` having a single parameter `period`.

Listing 1.3: Example `setParam()` implementation for fictional `BHV_YourBehavior`.

```
1  bool BHV_YourBehavior::setParam(string param, string value)
2  {
3      if(param == "period") {
4          double time_value = atof(value.c_str());
5          if((time_value < 0) || (!isNumber(value)))
6              return(false);
7          m_period = time_value;
8          return(true);
9      }
10     return(false);
11 }
```

Since the `period` parameter refers to a time period, a check is made on line 4 that the value component indeed is not a negative number. The `atof()` function on line 6, which converts an ASCII string to a floating point value, returns zero when passed a non-numerical string, therefore the `isNumber()` function is also used to ensure the string represented by `value` represents a numerical value. The `isNumber()` function is part of the IvP utility library. A behavior implementation of this function without sufficient syntax or semantic checking simply runs the risk that faulty parameters are not detected at the time of helm launch, or during dynamic updates. Solid checking in this function will reduce debugging headaches down the road.

1.4 Behavior Functions Invoked by the Helm

The `IvPBehavior` superclass implements a number of functions invoked by the helm on each iteration. Two of these functions are overloadable as described previously - the `onRunState()` and `onIdleState()` functions. The basic flow of calls to a behavior from the helm are shown in Figure 2. These are discussed in more detail later in the section, but the idea is to execute certain behavior functions based on the *activity state*, which may be one of the four states depicted.

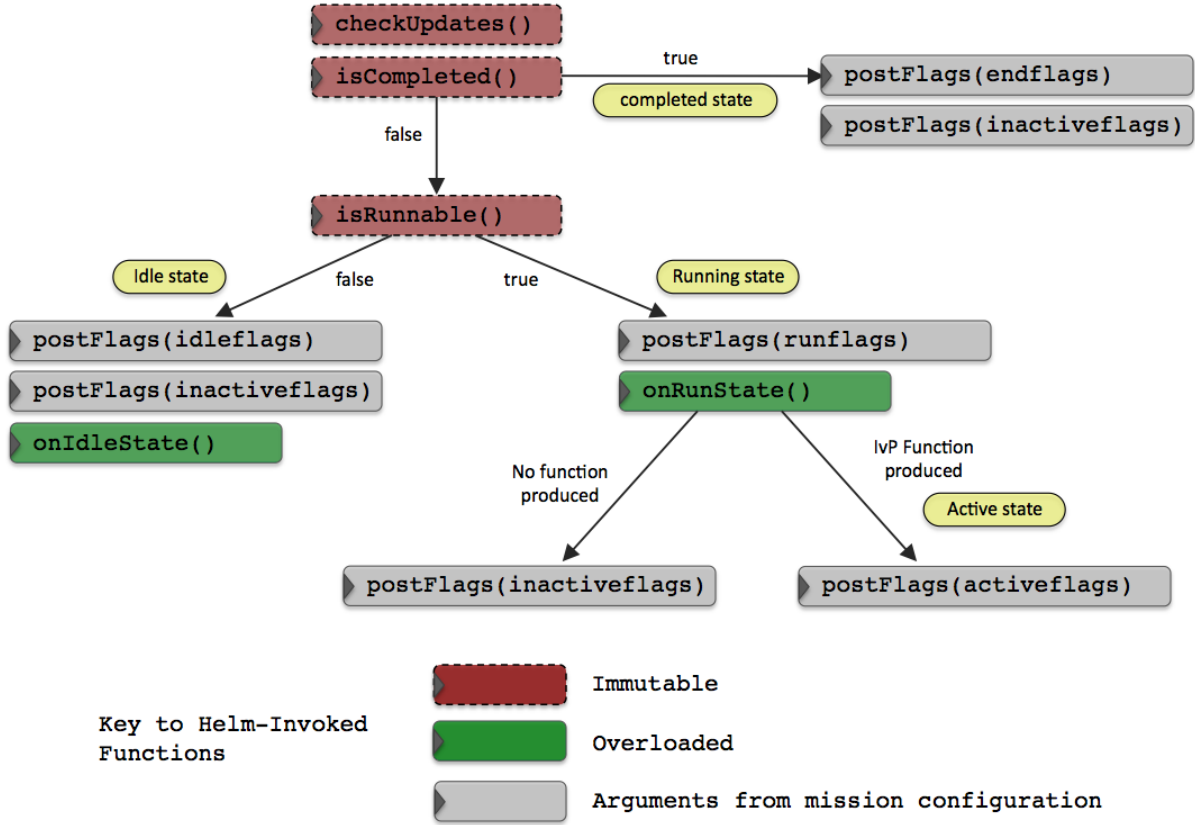


Figure 2: **Behavior function-calls by the helm:** The helm invokes a sequence of functions on each behavior on each iteration of the helm. The sequence of calls is dependent on what the behavior returns, and reflects the behaviors activity state. Certain functions are immutable and can not be overloaded by a behavior author. Two key functions, `onRunState()` and `onIdleState()` can be indeed overloaded as the usual hook for an author to provide the implementation of a behavior. The `postFlags()` function is also immutable, but the parameters (flags) are provided in the helm configuration (`*.bhv`) file.

An *idle* behavior is one that has not met its conditions for running. A *completed* behavior is one that has reached its objectives or exceeded its duration. A *running* behavior is one that has not yet completed, has met its run conditions, but may still opt not to produce any output. An *active* behavior is one that is running and is producing output in the form of an objective function.

The types of functions defined at the superclass level fall into one of the three categories below, only the first two of which are shown in Figure 2:

- Helm-invoked immutable functions - functions invoked by the helm on each iteration that the author of a new behavior may not re-implement.
- Helm-invoked overloadable functions - functions invoked by the helm that an author of a new behavior typically re-implements or overloads.
- User-invoked functions - functions invoked within a behavior implementation.

The user-invoked functions are utilities for common operations typically invoked within the implementation of the `onRunState()` and `onIdleState()` functions written by the behavior author.

1.4.1 Helm-Invoked Immutable Functions

These functions, implemented in the `IvPBehavior` superclass, are called by the helm but are *not* defined as virtual functions which means that attempts to overload them in a new behavior implementation will be ignored. See Figure 2 regarding the sequence of these function calls.

- `void checkUpdates()`: This function is called first on each iteration to handle requested dynamic changes in the behavior configuration. This needs to be the very first function applied to a behavior on the helm iteration so any requested changes to the behavior parameters may be applied on the present iteration. See Section 1.2.5 for more on dynamic behavior configuration with the `updates` parameter.
- `bool isComplete()`: This function simply returns a Boolean indicating whether the behavior was put into the *complete* state during a prior iteration.
- `bool isRunnable()`: Determines if a behavior is in the *running* state or not. Within this function call four things are checked: (a) if the `duration` is set, the duration time remaining is checked for timeout, (b) variables that are monitored for staleness are checked against (Section 1.2.9). (c) the run conditions must be met. (d) the behavior's decision domain (IvP domain) is a proper subset of the helm's configured IvP domain. See Section ?? for more detail on run conditions.
- `void postFlags(string flag-type)`: This function will post flags depending on whether the value of `flag-type` is set to "idleflags", "runflags", "activeflags", "inactiveflags", or "endflags". Although this function is immutable, not overloadable by subclass implementations, its effect is indeed mutable since the flags are specified in the mission configuration `*.bhv` file. See Section ?? for more detail on posting flags to the `MOOSDB` from the helm.

1.4.2 Helm-Invoked Overloaded Functions

These are functions called by the helm. They are defined as virtual functions so that a behavior author may overload them. Typically the bulk of writing a new behavior resides in implementing these three functions.

- `IvPFunction* onRunState()`: This function is called by the helm when deemed to be in the *running* state (Figure 2). The bulk of the work in implementing a new behavior is in this function implementation, and is the subject of Section 1.6.
- `void onIdleState()`: This function is called by the helm when deemed to be in the *idle* state (Figure 2). Many behaviors are implemented with this function left undefined, but it is a useful hook to have in many cases.
- `bool setParam(string, string)`: This function is called by the helm when the behavior is first instantiated with the set of parameter and parameter values provided in the behavior file. It is also called by the helm within the `checkUpdates()` function to apply parameter updates dynamically.

1.5 Local Behavior Utility Functions

The bulk of the work done in implementing a new behavior is in the implementation of the `onIdleState()` and `onRunState()` functions. The utility functions described below are designed to aid in that implementation and are generally "protected" functions, that is callable only from within the code of another function in the behavior, such as the `onRunState()` and `onIdleState()` functions, and not invoked by the helm.

1.5.1 Summary of Implementor-Invoked Utility Functions

The following is summary of utility functions implemented at the `IvPBehavior` superclass level.

- `void setComplete()`: The notion of what it means for a behavior to be "complete" is largely an issue specific to an individual behavior. When or if this state is reached, a call to `setComplete()` can be made and end flags will be posted, and the behavior will be permanently put into the *completed* state unless the `perpetual` parameter is set to true. See Section ?? for more on behavior run states.
- `void addInfoVars(string var_names)`: The helm will register for variables from the `MOOSDB` on a need-only basis, and a behavior is obligated to inform the helm that certain variables are needed on its behalf. A call to the `addInfoVars()` function can be made from anywhere with a behavior implementation to declare needed variables. This can be one call per variable, or the string argument can be a comma-separated list of variables. The most common point of invoking this function is within a behavior's constructor since needed variables are typically known at the point of instantiation. More on this issue in Section 1.5.3.
- `double getBufferDoubleVal(string varname, bool& result)`: Query the `info_buffer` for the latest (double) value for a given variable named by the string argument. The bool argument indicates whether the queried variable was found in the buffer. More on this in Section 1.5.2.
- `double getBufferStringVal(string varname, bool& result)`: Query the `info_buffer` for the latest (string) value for a given variable named by the string argument. The bool argument indicates whether the queried variable was found in the buffer. More on this in Section 1.5.2.
- `double getBufferCurrTime()`: Query the `info_buffer` for the current buffer local time, equivalent to the duration in seconds since the helm was launched. More on this in Section 1.5.2.
- `vector<double> getBufferDoubleVector(string var, bool& result)`: Query the `info_buffer` for all changes to the variable (of type double) named by the string argument, since the last iteration. The bool argument indicates whether the queried variable was found in the buffer. More on this in Section 1.5.2.
- `vector<string> getBufferStringVector(string var, bool& result)`: Query the `info_buffer` for all changes to the variable (of type string) named by the string argument, since the last iteration. The bool argument indicates whether the queried variable was found in the buffer. More on this in Section 1.5.2.

- `void postMessage(string varname, string value, string key)`: The helm can post messages (variable-value pairs) to the `MOOSDB` at the end of the helm iteration. Behaviors can request such postings via a call to the `postMessage()` function where the first argument is the variable name, and the second is the variable value. The optional `key` parameter is used in conjunction with the duplication filter and by default is the empty string. See Section ?? for more on the duplication filter.
- `void postMessage(string varname, double value, string key)`: Same as above except used when the posted variable is of type `double` rather than `string`. The optional `key` parameter is used in conjunction with the duplication filter and by default is the empty string. See Section ?? for more on the duplication filter.
- `void postBoolMessage(string varname, bool value, string key)`: Same as above, except used when the posted variable is a `bool` rather than `string`. The optional `key` parameter is used in conjunction with the duplication filter and by default is the empty string. See Section ?? for more on the duplication filter.
- `void postIntMessage(string varname, double value, string key)`: Same as `postMessage(string, double)` above except the numerical output is rounded to the nearest integer. This, combined with the helm's use of the *duplication filter*, can reduce the number of posts to the `MOOSDB`. The optional `key` parameter is used in conjunction with the duplication filter and by default is the empty string. See Section ?? for more on the duplication filter.
- `void postWMessage(string warning_msg)`: Identical to the `postMessage()` function except the variable name is automatically set to `BHV.WARNING`. Provided as a matter of convenience to the caller and for uniformity in monitoring warnings.
- `void postEMessage(string error_msg)`: Similar to the `postWMessage()` function except the variable name is `BHV.ERROR`. This call is for more serious problems noted by the behavior. It also results in an internal `state_ok` bit being flipped which results in the helm posting all-stop values to the actuators.
- `void postRepeatableMessage(string varname, string value)`: A convenience function. A posting of `postRepeatableMessage("F00", "bar")` is equivalent to `postMessage("F00", "bar", "repeatable")`.
- `void postRepeatableMessage(string varname, double value)`: A convenience function. A posting of `postRepeatableMessage("F00", 100)` is equivalent to `postMessage("F00", 100, "repeatable")`.

1.5.2 The Information Buffer

Behaviors do not have direct access to the `MOOSDB` - they don't read mail, and they don't post changes directly, but rather through the helm as an intermediary. The information buffer, or `info_buffer`, is a data structure maintained by the helm to reflect a subset of the information in the `MOOSDB` and made available to each behavior. This topic is hidden from a user configuring existing behaviors and can be safely skipped, but is an important issue for a behavior author implementing a new behavior. The `info_buffer` is a data structure shared by all behaviors, each behavior having an pointer to a single instance of the `InfoBuffer` class. This data structure is maintained by the helm, primarily

by reading mail from the `MOOSDB` and reflecting the change onto the buffer on each helm iteration, before the helm requests input from each behavior. Each behavior therefore has the exact same snapshot of a subset of the `MOOSDB`. A behavior author needs to know two things - how to ensure that certain variables show up in the buffer, and how to access that information from within the behavior. These two issues are discussed next.

1.5.3 Requesting the Inclusion of a Variable in the Information Buffer

A variable can be specifically requested for inclusion in the `info_buffer` by invoking the following function:

```
void IvPBehavior::addInfoVars(string varnames)
```

The string argument is either a single MOOS variable or a comma-separated list of variables. Duplicate requests are simply ignored. Typically such calls are invoked in a behavior's constructor, but may be done dynamically at any point after the helm is running. The helm will simply register with the `MOOSDB` for the requested variable at the end of the current iteration. Certain variables are registered for automatically on behalf of the behavior. All variables referenced in run conditions will be registered and accessible in the buffer. Variables named in the `updates` and `nostarve` parameters will also be automatically registered.

1.5.4 Accessing Variable Information from the Information Buffer

A variable value can be queried from the buffer with one of the following two function calls, depending on whether the variable is of type double or string.

```
string IvPBehavior::getBufferStringVal(string varname, bool& result)
double IvPBehavior::getBufferDoubleVal(string varname, bool& result)
```

The first string argument is the variable name, and the second argument is a reference to a Boolean variable which, upon the function return, will indicate whether the queried variable was found in the buffer. A duration value indicating the elapsed time since the variable was last changed in the buffer can be obtained from the following function call:

```
double IvPBehavior::getBufferTimeVal(string varname);
```

The string argument is the variable name. The returned value should be exactly zero if this variable was updated by new mail received by the helm at the beginning of the current iteration. If the variable name is not found in the buffer, the return value is -1. The "current" buffer time, equivalent to the cumulative time in seconds since the helm was launched, can be retrieved with the following function call:

```
double IvPBehavior::getBufferCurrTime()
```

The buffer time is a local variable of the `info_buffer` data structure. It is updated once at the beginning of the helm `OnNewMail()` loop prior to processing all new updates to the buffer from the MOOS mail stack, or at the beginning of the `Iterate()` loop if no mail is processed on the current

iteration. Thus the time-stamp returned by the above call should be exactly the same for successive calls by all behaviors within a helm iteration.

The values returned by `getBufferStringVal()` and `getBufferDoubleVal()` represent the latest value of the variable in the `MOOSDB` at the point in time when the helm began its iteration and processed its mail stack. The value may have changed several times in the `MOOSDB` between iterations, and this information may be of use to a behavior. This is particularly true when a variable is being posted in pieces, or a sequence of delta changes to a data structure. In any event, this information can be recovered with the following two function calls:

```
vector<string>  IvBehavior::getBufferStringVector(string varname, bool& result)
vector<double> IvBehavior::getBufferDoubleVector(string varname, bool& result)
```

They return all values updated to the buffer for a given variable since the last iteration in a vector of strings or doubles respectively. The latest change is located at the highest index of the vector. An empty vector is returned if no changes were received at the outset of the current iteration.

1.6 Overloading the `onRunState()` and `onIdleState()` Functions

The `onRunState()` function is declared as a virtual function in the `IvPBehavior` superclass intended to be overloaded by the behavior author to accomplish the primary work of the behavior. The primary behavior output is the objective function. This is what drives the vehicle. The objective function is an instance of the class `IvPFunction`, and a behavior generates an instance and returns a pointer to the object in the following function:

```
IvPFunction* onRunState()
```

This function is called automatically by the helm on the current iteration if the behavior is deemed to be in the *running* state, as depicted in Figure 2. The invocation of `onRunState()` does not necessarily mean an objective function is returned. The behavior may opt not to for whatever reason, in which case it returns a null pointer. However, if it does generate a function, the behavior is said to be in the *active* state. The steps comprising the typical implementation of the `onRunState()` implementation can be summarized as follows:

- Get information from the `info_buffer`, and update any internal behavior state.
- Generate any messages to be posted to the `MOOSDB`.
- Produce an objective function if warranted.
- Return.

The same steps hold for the `onIdleState()` function except for producing an objective function. The first two steps have been discussed in detail. Accessing the `info_buffer` was described in Sections 1.5.2 - 1.5.4. The functions for posting messages to the `MOOSDB` from within a behavior were discussed in Section 1.5.1. Further issues regarding the posting of messages were covered in Section ?? The remaining issue to discuss is how objective functions are generated. This is covered in the `IvPBuild` Toolbox documentation.

1.7 Dynamic Behavior Spawning

In certain scenarios it may not be practical or possible to know in advance all the behaviors needed to accomplish mission objectives. For example, if the helm uses a certain kind of behavior to deal with another vehicle in its operation area, for collision avoidance or trailing etc., the identities or number of such vehicles may not be known when the mission planner is configuring the helm's behavior file. One way to circumvent this problem is to design a collision avoidance behavior, for example, to handle all known contacts. However, this has a couple drawbacks. It would entail a degree of multi-objective optimization be implemented within the behavior to produce an objective function that was comprehensive across all contacts. This would likely be much more computationally expensive than simply generating an objective function for each contact. It also may be advantageous to have different types of collision avoidance behaviors for different contact types or collision avoidance protocols. In any event, the helm support for dynamic behavior spawning gives behavior architects and mission planners another potentially powerful option for implementing an autonomy system.

1.7.1 Behavior Specifications Viewed as Templates

The `templating` parameter may be used to turn an otherwise static behavior specification into a template for spawning new behaviors dynamically after the helm has been launched. Instantiation requests are received via the `updates` parameter described in Section 1.2.5. Updates received through this variable are normally used to change behavior parameters dynamically, but they can be further used to request the spawning of a new behavior by including the following component:

```
name = <new-behavior-name>
```

If the `<new-behavior-name>` is not the name of the behavior given in the behavior specification, and if it is not the name of a behavior already presently instantiated by the helm, the helm interprets this as a request to spawn a new behavior, if templating is enabled. Templating is enabled by including the following component in the behavior specification:

```
templating = <templating-mode>
```

The `<templating-mode>` may be set to either "disallowed" (the default), "clone", or "spawn". In the "clone" mode, the helm will instantiate a behavior immediately upon helm startup. In the "spawn" mode, the helm will not instantiate a behavior until it receives a request to do so via the `updates` parameter as described above. An example of a behavior configured to allow dynamic spawning is given in Listing ??, taken from the Berta example mission.

For a behavior configured with templating enabled in the "spawn" mode, the helm will *not* spawn a behavior at the helm startup time. However, internally it will indeed spawn such a behavior, check that it can be found and built as configured, and then destroy it immediately. This means that the behavior configuration found in the `.bhv` configuration file must not have an invalid configuration. It is preferable to know at helm launch time that a behavior is misconfigured, rather than waiting for the spawning event to occur perhaps hours into a mission and being surprised that a critical behavior, such as collision avoidance, failed to be spawned.

1.7.2 Behavior Completion and Removal from the Helm

All behaviors, whether statically spawned upon helm startup, or dynamically spawned during the mission, are capable of dying and being removed from the helm. Death and removal are part of the consequences of a behavior entering the `completed` state. Behavior run states were discussed in Section ???. A completed behavior configured with `perpetual=true` will not die upon completion. Once a behavior dies, its name is removed from the helm's internal registry of currently-spawned behaviors and a new behavior by the same name may be spawned at a future time.

1.7.3 Example Missions with Dynamic Behavior Spawning

Two example missions are provided that demonstrate the workings of dynamic behavior spawning, The Echo mission in Section ??, and the Berta mission in Section ??. The Echo mission involves a single vehicle with its helm configured to spawn dynamic behaviors of the type `BHV_BearingLine`. These behaviors do nothing more than post a viewable line segment to the `MOOSDB` between ownship and a point in the operation area. The interesting thing about this example is that the mission is configured with an event script (via `uTimerScript`) to automatically cue the spawning of 5000 behaviors over about one hour. Each behavior has a random duration of less than a minute, so behaviors are spawning and dying quite rapidly with visual confirmation via the viewable line segments.

The second example mission, the Berta mission, involves two vehicles that are loitering near one another. Periodically their loiter assignments are randomly altered (again through `uTimerScript`). The change in loiter locations repeatedly puts them on an unpredictable and random near-collision course and each vehicle needs to spawn a collision avoidance behavior. The interesting thing about this scenario is that the behavior, the `BHV_AvoidCollision` behavior, is an actual behavior of common use, unlike the `BHV_BearingLine` behavior used in the Echo mission. This example also uses the `pBasicContactMgr` to coordinate the receiving of contact reports with helm behavior spawning.

1.7.4 Examining the Helm's Life Event History

Behavior spawning, and behavior completion and removal from the helm, are two types of *life events* the helm takes note of and posts in the MOOS variable `IVPHELM_LIFE_EVENT`. A third type of life event occurs when a behavior spawning is aborted due to either a syntax error or a name collision. Monitoring life events at run time is possible by scoping on the variable `IVPHELM_LIFE_EVENT` with either `uXMS` or `uMS`. A better method is available via the `uHelmScope` application. It automatically registers for the `IVPHELM_LIFE_EVENT` variable and will generate a formatted report like that shown in Listing 4. In the post-mission analysis phase, the `aloghelm` application may be used to examine the life event history and will generate the same formatted report from a given alog file.

Listing 1.4: A Life Event History generated with either the `uHelmScope` or `aloghelm` utilities.

```
1 *****
2 *      Summary of Behavior Life Events      *
3 *****
4
5 Time    Iter  Event  Behavior      Behavior Type      Spawning Seed
```

6	-----	----	-----	-----	-----
7	47.84	1	spawn	loiter	BHV_Loiter helm startup
8	47.84	1	spawn	waypt_return	BHV_Waypoint helm startup
9	47.84	1	spawn	station-keep	BHV_StationKeep helm startup
10	101.79	218	spawn	avd_henry	BHV_AvoidCollision name=avd_henry#contact=henry
11	161.20	423	death	avd_henry	BHV_AvoidCollision
12	297.07	969	spawn	avd_henry	BHV_AvoidCollision name=avd_henry#contact=henry
13	351.80	1159	death	avd_henry	BHV_AvoidCollision
14	461.37	1599	spawn	avd_henry	BHV_AvoidCollision name=avd_henry#contact=henry
15	516.51	1795	death	avd_henry	BHV_AvoidCollision
16	644.94	2311	spawn	avd_henry	BHV_AvoidCollision name=avd_henry#contact=henry
17	704.31	2517	death	avd_henry	BHV_AvoidCollision
18	730.02	2620	abort		BHV_AvoidCollision name=avd_henry#foo=bar
19	825.17	3002	spawn	avd_henry	BHV_AvoidCollision name=avd_henry#contact=henry

The life event history shown in Listing 4 was taken from the Berta example mission, the "gilda" vehicle, described in Section ?? . The time-stamp reported in column one is the elapsed time between the event and the time of the helm's startup. The first three events, in lines 6-8, reflect the three static behaviors spawned when the helm was launched, in first iteration of the helm. The collision avoidance behavior was spawned each time (lines 9, 11, 13 etc.) the vehicle "henry" came within sufficiently close range. Each time the "henry" vehicle passed and opened range to a sufficient amount, the collision avoidance behavior completed and died (lines 10, 12, 14, etc.). Line 17 shows an example of an aborted spawning. This was brought about purposely by poking the MOOSDB with the Spawning Seed shown for that line. Since the collision avoidance parameter does not have a parameter "foo", the spawning failed.

Accessing the life event history via `uHelmScope` may be done by launching the scope with the vehicle's mission file, and hitting the 'L' key to toggle into the life event history mode. Or one may launch the scope directly into this mode via:

```
uHelmScope --life targ_gilda.moos}
```

The same summary may also be accessed after mission completion via the log files:

```
aloghelm --life gilda.alog
```

Note that perhaps not all life events will be displayed when using `uHelmScope`, depending on when it is launched relative to `pHelmIvP`. When `uHelmScope` connects to the MOOSDB it will only receive the latest and all following posts to the variable `IVPHELM.LIFE.EVENT`. If `uHelmScope` connects after `pHelmIvP` is launched and put into drive, it may have missed older postings. The initial spawning events do not occur in the helm until the helm enters the DRIVE state. (See Section ?? about helm state). In the example in Listing 4, the helm apparently was in the PARK state for about 48 seconds before it was put into drive and began to execute its first iteration. The full event history should always be accessible via the log file however.